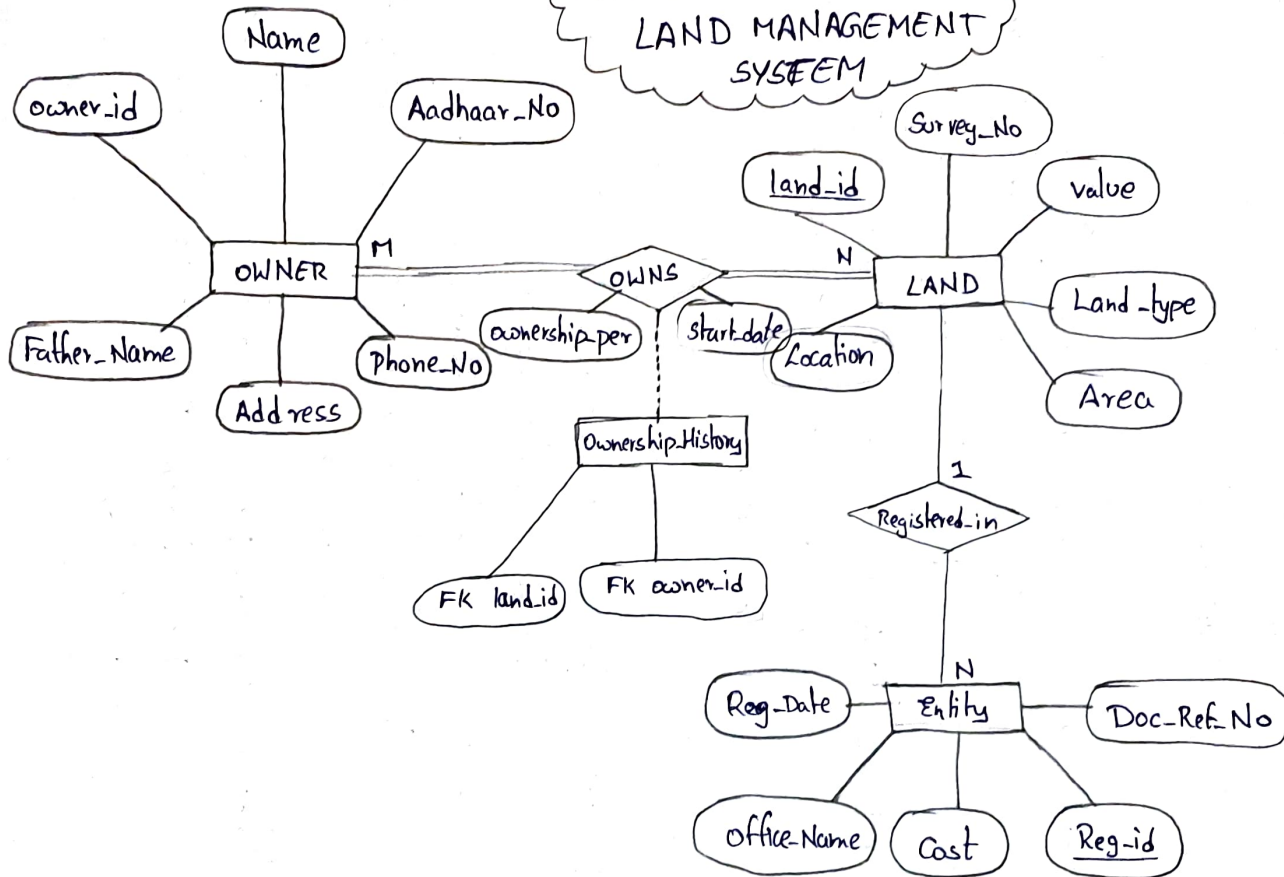


# ER DIAGRAM FOR LAND MANAGEMENT SYSTEM



- \* Owner: stores personal details of the landholder
- \* LAND: stores physical property data
- \* RegistrationEntity: stores legal filing data
- \* ownership-history: A record-keeping entity that tracks previous & current landholders

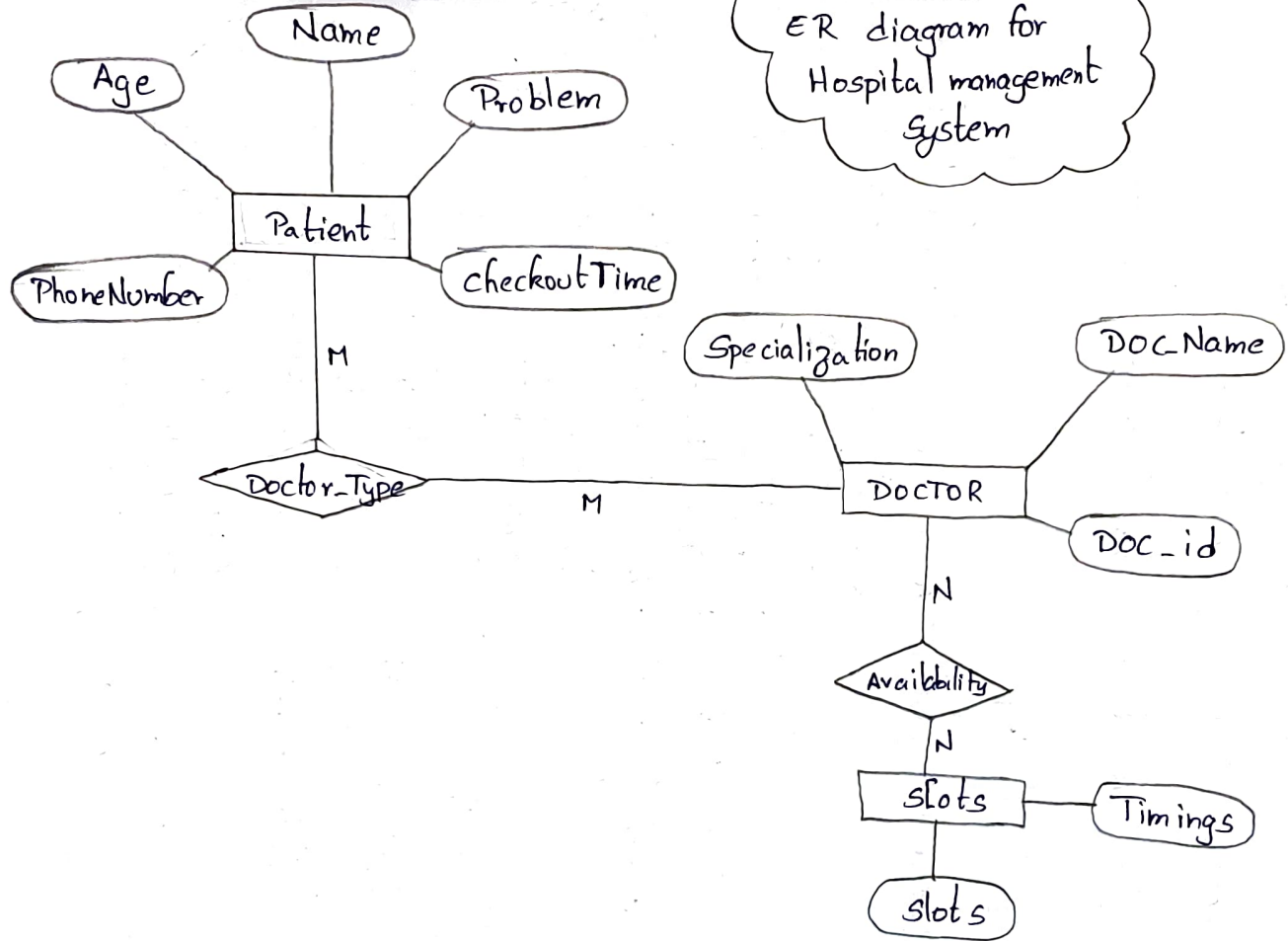
### Relationships:-

- \* OWNS (M:N): A many to many link showing that multiple owners can hold shares in multiple lands. It tracks specific details like ownership-per
- \* Registered-in (1:N): A one to many link indicating that one registration record/office handles multiple land entries.

### Key features:-

- \* Unique Identifiers: Uses owner-id, land-id & Reg-id as primary keys to ensure data uniqueness.
- \* Audit Trail:- The ownership-his & start-date attributes allow the system to track the "chain of custody" for any piece of land.
- \* Legal validation:- Connects physical land dimensions with legal documentation (Doc-Ref-No) for government verification

ER diagram for  
Hospital management  
system



### 3. Core Entities:-

- \* Patient: Stores data for individuals seeking care
- \* Doctors: Stores professional data
- \* Slots: A dedicated entity to manage time intervals

### 2. Key Relationship:-

- \* Doctor\_Type (M:M) : A many to many relationship between patients & Doctors. Multiple patients can consult multiple types of doctors. It includes the descriptive attribute CheckoutTime.
- \* Availability (N:N) : A relationship linking doctors to specific time slots, allowing the system to track when a doctor is free to see patients.

### 3. Logical Flow:-

Identification: DocId likely serves as the primary key for doctor entity to ensure records

Specialization Mapping:- The diagram links specialization directly to doctor, ensuring patients with a specific problem are matched with correct medical expert

## Relational Algebra :-

Relational algebra is used in DBMS to manipulate relations

These are divided into two categories:

- i) Unary operations
- ii) Binary operations

Unary operations contains selection, projection & Rename  
Binary operations contains set operations, joins & Division

### 1, Selection ( $\sigma$ ) :-

The selection operation filters rows based on specific condition  
It is denoted as " $\sigma$ "

Example :-

$\sigma_{\text{location} = \text{"Vijayawada"}} (\text{Land})$

$\sigma_{\text{aadhaar\_No} = \text{"123456789012"}} (\text{Owner})$

$\sigma_{\text{value} > 500000} (\text{Land})$

### 2, Projection ( $\pi$ ) :-

The projection operation selects specific columns from a table to show the data. It is denoted as " $\pi$ ".

Example :-  $\pi_{\text{name, phone\_no}} (\text{Owner})$

$\pi_{\text{location, value}} (\text{Land})$

$\pi_{\text{doc-ref-no, reg-date}} (\text{Registrations})$

### 3, Rename (P) :-

The rename operation allows us to change the name of a relation or its attributes. It is denoted as "P".

Example:- P 0 (owner)

P (owner\_name, phone) ( $\pi$  name, phone-no (owner))

P 1 (Land)

### 4, Set operations:-

These are divided into four Union, Intersection, set difference & Cross product

#### Union (U):-

Union combines all rows from two relations

Example:-  $\pi$  name ( $\sigma_{\text{address} = \text{"Vijayawada"}}(\text{owner})$ )

$\cup$   
 $\pi$  name ( $\sigma_{\text{address} = \text{"Guntur"}}(\text{owner})$ )

#### Intersection (n):-

It gives the common data from two tables

Example:-  $\pi$  owner\_id ( $\sigma_{\text{location} = \text{"Vijayawada"}}(\text{owner})$ )

$\cap$   
 $\pi$  owner\_id ( $\sigma_{\text{location} = \text{"Guntur"}}(\text{owner})$ )

#### Set difference (-):-

It gives the common operation finds rows that are present in one table but not in second table

Example:-  $\pi$  owner\_id ( $\sigma_{\text{location} = \text{"Vijayawada"}}(\text{Ownership} \bowtie \text{Land})$ )

$\bar{\pi}$  owner\_id ( $\sigma_{\text{location} = \text{"Guntur"}}(\text{Ownership} \bowtie \text{Land})$ )

## Cartesian Product ( $\times$ ):-

It is also known as cross product. This operation combines every row of the first table with every row of the second table.

Example Land  $\times$  owner

## Joins ( $\bowtie$ ):-

Joins operations are used to combine two or more relations

### \* Natural join ( $\bowtie$ ):-

It automatically joins on common attributes

Example:- ownership  $\bowtie$  owner

### \* Theta join ( $\theta$ ):-

Example:- ownership  $\bowtie$  ownership.owner\_id = owner.owner\_id owner

### \* Equi join

It join using equality condition

ownership  $\bowtie$  ownership.owner\_id = owner.owner\_id owner

## Division ( $\div$ ):-

It is used for queries that involve the keyword "all". It returns rows from table A that are associated with every row in table B

Example:-  $\pi_{\text{owner\_id}}(\text{ownership}) / \pi_{\text{land\_id}}(\text{Land})$